Module 4 - Embeddings & Semantic Search (3-Line Revision Notes)

Understanding Embeddings

• Embeddings convert text into dense numerical vectors.

• Similar meanings produce vectors that are close in vector space.

• They are the foundation of semantic search and RAG systems.

Embedding Models

• Embedding models generate vector representations of text.

• Use the same model for both documents and queries.

• Examples include nomic-embed-text, bge, and sentence-transformers models.

Similarity Metrics

• Similarity metrics compare the closeness of vectors.

• Common metrics are Cosine Similarity, Dot Product, and Euclidean Distance.

• The correct metric depends on how the embedding model was trained.

Cosine Similarity

• Measures the angle between two vectors.

• Values closer to 1 indicate higher semantic similarity.

• Most text embedding models recommend cosine similarity.

Dot Product

• Computes the product of vector magnitudes and directions.

• After L2 normalization it becomes equivalent to cosine similarity.

• Often used with IndexFlatIP in FAISS.

Euclidean Distance

• Measures the straight-line distance between vectors.

• Smaller distances indicate better matches.

• Used by IndexFlatL2 in FAISS.

Interpreting Similarity Scores

• Higher cosine or dot-product scores indicate stronger matches.

• Lower Euclidean distances indicate stronger matches.

• Compare scores only within the same similarity metric.

Semantic Search

• Semantic search retrieves information by meaning rather than keywords.

• It understands synonyms and related concepts.

• It is widely used in chatbots and enterprise search.

Embedding a Document Corpus

• Convert every document or chunk into an embedding once.

• Store the embeddings for future searches.

• Re-embed only when the source content changes.

Storing Embeddings

• Embeddings can be stored in files or vector databases.

• Avoid regenerating embeddings on every application start.

• Store metadata separately when using FAISS.

Storing as NumPy Arrays

• Convert embeddings to NumPy arrays with float32.

• This is the expected format for FAISS.

• NumPy provides efficient memory usage and fast computation.

Embedding the User Query

• Convert each user query into an embedding before searching.

• Always use the same embedding model as the documents.

• The query embedding is generated for every search request.

Top-K Retrieval

• Retrieve only the K most relevant results.

• A smaller K reduces tokens sent to the LLM.

• Typical values range from 3 to 10 depending on the use case.

Minimal Semantic Search with FAISS

• Generate embeddings, build a FAISS index, and search it.

• FAISS returns indices of the closest vectors.

• Use the indices to retrieve the original documents.

Hybrid Search

• Hybrid search combines BM25 keyword search with vector search.

• It improves retrieval for both exact terms and semantic meaning.

• This is the preferred approach in many production RAG systems.

Key Parameters

Embedding: Numerical representation of text for semantic comparison.

Embedding Dimension: Length of each embedding vector (e.g., 768 or 1024).

Vector: Array of floating-point values representing meaning.

Cosine Similarity: Higher value = better semantic match.

Dot Product: Higher value = better match; normalize for cosine equivalence.

Euclidean Distance: Lower value = better match.

Top-K: Number of search results returned.

Similarity Threshold: Minimum score required to accept a match.

Document Embedding: Vector created from a document or chunk.

Query Embedding: Vector created from the user's query.

NumPy float32: Recommended datatype for FAISS.

Shape (n,d): n = embeddings, d = dimensions.

FAISS: High-performance vector similarity search library.

BM25: Keyword ranking algorithm.

Hybrid Search: Combines BM25 and vector search.

Metadata: Stores source, page, title and other document details.

Vector Index: Data structure optimized for vector retrieval.